

02. Travel Agent - Setup del progetto

1. Setup del Progetto: Una Guida Dettagliata

1.1 Creazione dell'Ambiente di Sviluppo

Poetry: Gestione delle Dipendenze

Poetry è uno strumento moderno per la gestione delle dipendenze e del packaging in Python. Ecco perché lo utilizziamo:

1. Creazione del Progetto:

```
# Inizializza un nuovo progetto Poetry
poetry new travel-agent-api

cd travel-agent-api
```

alcune dipendenze che ci serviranno al giorno attuale richiedono un python compreso tra **<4.0,>=3.9.0**

quindi e' necessario modificare il file pyproject.toml specificando il python richiesto (requires-python), in questo modo

```
requires-python = "<4.0,>=3.9.0"
```

Installazione delle Dipendenze

Analizziamo ogni dipendenza e il suo scopo:

```
poetry add fastapi uvicorn openai langchain-openai langchain-community langgraph pydantic
python-dotenv google-search-results
```

- **fastapi**: Framework web moderno per costruire API
- **uvicorn**: Server Web ASGI per FastAPI
- **openai**: Client ufficiale per l'API di OpenAI
- **langchain-openai**: Integrazioni OpenAI per LangChain
- **langchain-community**: Componenti community di LangChain
- **langgraph**: Per la creazione di agenti conversazionali
- **pydantic**: Validazione dei dati e serializzazione
- **python-dotenv**: Gestione delle variabili d'ambiente
- **google-search-results**: Client per SerpApi

1.2 Struttura del Progetto

Usa ls o tree per visualizzare la struttura del nuovo progetto poetry

```
$ tree

.
├── README.md
├── poetry.lock
└── pyproject.toml
```

```
├── tests
│   └── __init__.py
└── travel_agent_api
    └── __init__.py
```

3 directories, 5 files

Mi raccomando, documentate tutti i passaggi importanti sul file README.md e in modo particolare le modalità di utilizzo del sistema.

Il `poetry.lock` e il `pyproject.toml` sono due file fondamentali per la gestione delle dipendenze di un progetto Python usando Poetry.

- `pyproject.toml` : È il file di configurazione principale del progetto. Contiene i metadati del progetto (nome, versione, autore, ecc.), le dipendenze richieste e le configurazioni degli strumenti di build.
- `poetry.lock` : Questo file viene generato automaticamente da Poetry e contiene le versioni esatte di tutte le dipendenze installate, incluse le dipendenze indirette. Garantisce che tutti gli sviluppatori del progetto utilizzino esattamente le stesse versioni dei pacchetti, assicurando la riproducibilità dell'ambiente di sviluppo.

In prospettiva, alla fine del progetto, questa sarà la struttura finale. Ma ci arriveremo passo passo

```
travel-agent-api/
├── app/                                # Directory principale dell'applicazione
│   ├── __init__.py                    # Rende app un modulo Python
│   ├── main.py                        # Punto di ingresso dell'applicazione
│   └── routes/                        # Gestione delle route API
│       ├── __init__.py
│       └── chat_router.py             # Route per la chat dell'agente
│   └── services/                     # Logica di business
│       └── agent_service.py           # Servizio principale dell'agente
│   └── tools/                        # Strumenti dell'agente
│       ├── flights_finder.py          # Tool per la ricerca voli
│       ├── hotels_finder.py           # Tool per la ricerca hotel
│       ├── chain_historical_expert.py # Tool per info storiche
│       └── chain_travel_plan.py        # Tool per pianificazione
├── .env                              # Variabili d'ambiente (non versionato)
└── .env.example                      # Template per .env
```

2. Configurazione dell'Ambiente

2.1 Configurazione delle Variabili d'Ambiente

I file che iniziano con un punto (.) sono chiamati file nascosti o "dotfile" nei sistemi Unix e Linux. Questi file sono nascosti per default quando si visualizza una directory, per evitare di mostrare file di configurazione che normalmente non necessitano di essere modificati frequentemente. Per visualizzarli è necessario utilizzare il comando `ls -a`. Un esempio particolare di questi file nascosti è il `.env`, che viene utilizzato per memorizzare variabili d'ambiente e informazioni sensibili come chiavi API, credenziali di database o altre configurazioni specifiche dell'ambiente di sviluppo. Il file `.env` è particolarmente importante nello sviluppo di applicazioni moderne perché permette di separare le configurazioni sensibili dal codice sorgente, migliorando la sicurezza e facilitando la gestione di diverse configurazioni tra ambienti di sviluppo, test e produzione. È una pratica comune escludere il file `.env` dal controllo versione (ad esempio attraverso `.gitignore`) e fornire invece un file `.env.example` come template per gli altri sviluppatori.

Crea ora il file `.env.example`

. Puoi usare VSCode per creare un nuovo file vuoto, oppure da terminale il comando `touch .env.example`

Il file `.env.example` serve come template e documenta le variabili d'ambiente necessarie, incolla quello che vedi nel riquadro seguente:

```
OPENAI_API_KEY="YOUR_OPENAI_API_KEY"      # Chiave API di OpenAI
SERPAPI_API_KEY="YOUR_SERPAPI_API_KEY"     # Chiave API di SerpApi per voli/hotel
```

2.2 Setup del File .env

ora possiamo creare una copia del file `.env.example` per ottenere il file effettivo `.env`

```
# Copia il template
cp .env.example .env

# Modifica il file .env con le tue chiavi
code .env
```

Per ottenere le chiavi API necessarie:

1. OpenAI API Key

1. Registrati su <https://platform.openai.com>
2. vai su <https://platform.openai.com/settings/organization/api-keys>
3. Clicca sul bottone verde "Create new secret key"
4. dai un nome alla chiave e seleziona il project di default
5. copia la chiave

2. SerpApi Key

1. Registrati su <https://serpapi.com>
2. vai su <https://serpapi.com/dashboard>
3. clicca l'icona copia della chiave di default già creata in automatico dal sistema

2.3 Sicurezza

È importante:

- Non fare commit mai del file `.env` nel repository
- Aggiungere `.env` al `.gitignore`
- Mantenere le chiavi API sicure e non condividerle
- Ruotare regolarmente le chiavi API in produzione

Se questi dati finissero nel repository Git pubblicamente accessibile, potrebbero essere sfruttati da malintenzionati per:

3. Accedere ai tuoi servizi e database senza autorizzazione
4. Utilizzare le tue chiavi API per fare richieste a tuo nome, potenzialmente causando costi elevati
5. Compromettere la sicurezza dell'intera applicazione
6. Rubare dati sensibili degli utenti

La sicurezza delle chiavi API e delle credenziali è un aspetto critico dello sviluppo software moderno. Una singola chiave compromessa può portare a violazioni dei dati, perdite finanziarie e danni alla reputazione.

2.4 Modalità Test/Produzione

La variabile `API_MODE` :

- `test` : Usa dati di esempio per lo sviluppo

- `prod` : Usa le API reali per le richieste



Questo setup fornisce una base solida per lo sviluppo dell'applicazione, con una struttura chiara e una gestione sicura delle configurazioni sensibili.